DA5402 Project Report
Sales Forecasting System with MLOps

Rishika Samyam
CE22B101

DA5402: MLOps

Indian Institute of Technology Madras

Contents

1 Abstract **3**

2 Introduction **3**

3 Problem Statement **3**
- 3.1 Business Objective . . . 4
- 3.2 Machine Learning Objective . . . 4
- 3.3 Operational Objective . . . 4

4 Success Metrics **4**
- 4.1 ML Metrics . . . 4
- 4.2 Operational Metrics . . . 5

5 System Overview **5**
- 5.1 Offline Layer . . . 5
- 5.2 Online Layer . . . 5
- 5.3 Observability Layer . . . 6

6 Architecture **6**
- 6.1 Architecture Diagram . . . 6
- 6.2 Architecture Explanation . . . 6

7 Dataset and Processing **7**
- 7.1 Relevant Fields . . . 7
- 7.2 Feature Categories . . . 8

8 Offline ML Pipeline **8**
- 8.1 Stage 1: Data Preparation . . . 8
- 8.2 Stage 2: Model Training . . . 8
- 8.3 Stage 3: Model Evaluation . . . 9
- 8.4 DVC Pipeline Representation . . . 9

9 Model Development and Selection **9**
- 9.1 Selection Strategy . . . 9

10 Pipeline Visualization **9**
- 10.1 Airflow DAG . . . 9

11 Experiment Tracking with MLflow **10**
- 11.1 Tracked Information . . . 10
- 11.2 Role in Reproducibility . . . 10

12 Deployment and Packaging **10**
12.1 Containers 10
12.2 Environment Parity 11

13 API Design **11**
13.1 Main API Endpoints 11

14 Frontend Design **12**

15 Monitoring and Alerting **12**
15.1 Grafana Dashboard 13
15.2 Prometheus and Alerts 13

16 Drift Detection and Feedback Loop **13**
16.1 Drift Detection 13
16.2 Feedback Loop 14

17 Testing and Validation **14**
17.1 Testing Approach 14
17.2 Acceptance-Oriented Validation 14

18 Conclusion **14**

A Appendix A: Important URLs **15**

B Appendix B: AI Usage Disclosure **15**

1 Abstract

This report presents an end-to-end machine learning system for forecasting daily Rossmann store sales using a Docker-based, local-first MLOps stack. The project combines data preparation, feature engineering, model training, experiment tracking, model selection, online inference, monitoring, feedback collection, and reproducible orchestration into a unified application. The final system includes a user-facing web interface, a decoupled FastAPI-based serving layer, DVC-managed offline pipeline stages, MLflow-backed experiment tracking, Prometheus instrumentation, Grafana dashboards, and an Airflow DAG for offline pipeline visualization. The design emphasizes reproducibility, operational observability, modular deployment, and clarity for both technical evaluators and non-technical end users.

2 Introduction

Retail sales forecasting is a practical machine learning problem with clear business impact. A store manager benefits from reliable estimates of expected daily revenue, especially when planning inventory, staffing, promotions, and holiday operations. In this project, the chosen task is daily sales forecasting for Rossmann stores using historical sales records, store metadata, promotion indicators, and temporal features.

The objective is not only to train a predictive model, but to package the complete lifecycle into a maintainable MLOps system. Accordingly, the project was designed as a productized ML application rather than as a standalone notebook workflow. The final solution includes:

- a reproducible offline ML pipeline,
- experiment tracking and selected-model metadata,
- a separate online inference stack,
- user-facing forecast and feedback workflows,
- runtime monitoring and alert visibility,
- and orchestration support for pipeline visualization.

3 Problem Statement

The problem addressed in this work is the prediction of daily sales for Rossmann stores. The system receives store-specific and date-specific inputs and returns predicted sales, along with operational metadata such as inference latency, selected model run ID, and a request-level drift flag.

3.1 Business Objective

The core business objective is to support store planning decisions through reliable short-horizon demand estimates. These predictions can help with:

- staff planning,
- inventory preparation,
- promotion scheduling,
- and exception handling for unusual dates.

3.2 Machine Learning Objective

The machine learning objective is to learn a mapping from historical sales and contextual variables to next-day store sales.

3.3 Operational Objective

The operational objective is to deploy the forecasting model as a monitored, reproducible, and demonstrable ML system with:

- containerized deployment,
- clear service separation,
- runtime metrics,
- offline pipeline traceability,
- and user-friendly interaction.

4 Success Metrics

The project tracks both ML and system-level objectives.

4.1 ML Metrics

- RMSPE
- RMSE
- MAE
- MAPE
- R^2

4.2 Operational Metrics

- request throughput,
- request latency,
- failure counts,
- service reachability,
- request-level drift events,
- feedback volume and live error summary.

5 System Overview

The system is organized into offline and online components.

5.1 Offline Layer

The offline layer handles:

- raw data ingestion,
- data preparation,
- feature engineering,
- baseline-statistics generation,
- model training,
- model comparison,
- and evaluation report generation.

5.2 Online Layer

The online layer handles:

- user forecast requests,
- prediction serving,
- response metadata,
- request logging,
- drift checks,
- and feedback capture.

5.3 Observability Layer

The observability layer provides:

- exporter-based runtime metrics,
- dashboard-level monitoring views,
- drift visibility,
- latency tracking,
- and service availability insight.

6 Architecture

The project uses a modular architecture with clean separation between frontend, gateway, model serving, pipeline orchestration, and monitoring.

6.1 Architecture Diagram

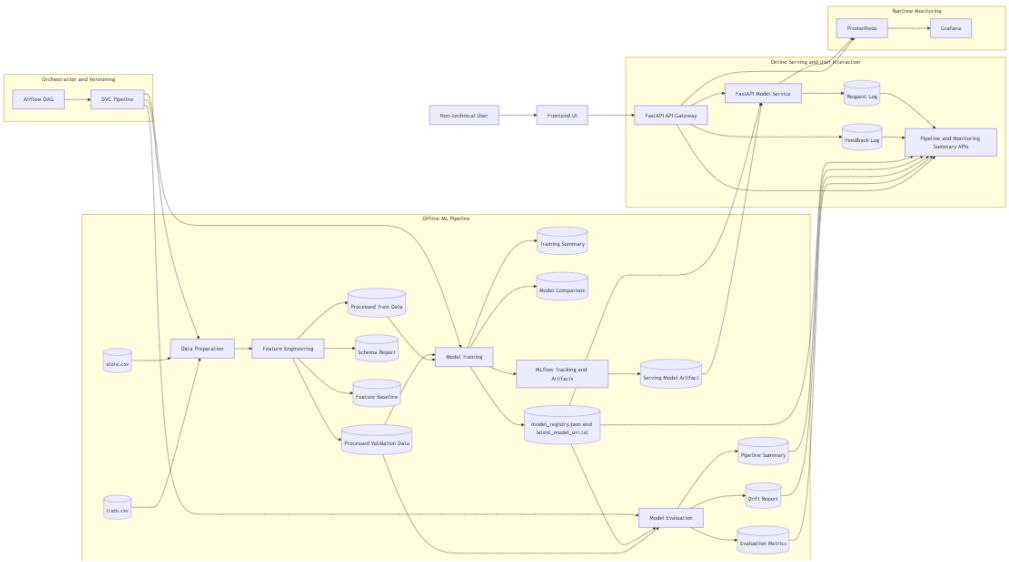

Figure 1: System Architecture

6.2 Architecture Explanation

The key architectural blocks are:

- **Frontend UI**: the user-facing portal for forecast generation, pipeline viewing, monitoring summary, and user manual access.

- **API Gateway**: the public-facing backend that validates requests, communicates with the model service, records feedback, and exposes pipeline/monitoring summary endpoints.
- **Model Service**: the inference backend that loads the selected model artifact and returns predictions with drift and latency metadata.
- **Offline ML Pipeline**: data preparation, feature engineering, training, and evaluation stages.
- **DVC**: reproducible stage management for offline pipeline execution.
- **MLflow**: experiment tracking and model artifact management.
- **Airflow**: DAG-based visualization and orchestration interface for offline pipeline stages.
- **Prometheus**: time-series metrics collection from the serving stack.
- **Grafana**: visualization layer for operational metrics and alerts.

7 Dataset and Processing

The project uses Rossmann sales data along with store metadata. The primary data sources are:

- `train.csv`
- `store.csv`

7.1 Relevant Fields

Examples of useful variables include:

- Store ID,
- Day of Week,
- Date,
- Promo,
- State Holiday,
- School Holiday,
- Store Type,
- Assortment,
- Competition Distance,

- Promo2 and related fields.

The raw files are converted into processed train and validation feature sets. During this transformation, schema-level summaries and feature baselines are also generated to support reproducibility and drift detection.

7.2 Feature Categories

The engineered feature space includes:

- **Calendar features**: Year, Month, Day, Week of Year, Month Start, Month End, Weekend indicator
- **Store features**: Store Type, Assortment, Competition Distance
- **Promotion features**: Promo, Promo2, Promo2Active
- **Competition features**: Competition opening month, year, and derived competition-open duration

Retail sales are strongly influenced by temporal and business-context signals. Directly using only raw columns would underrepresent important relationships. The engineered variables help the model capture:

- seasonal structure,
- campaign-related changes,
- long-running competition effects,
- and store-level business differences.

8 Offline ML Pipeline

The offline pipeline is organized into three major stages.

8.1 Stage 1: Data Preparation

This stage ingests raw training and store metadata, performs joins, handles missing values, prepares data types, and produces processed train and validation datasets.

8.2 Stage 2: Model Training

This stage trains candidate models using the processed datasets, compares validation performance, logs experiments, and writes selected model metadata.

8.3 Stage 3: Model Evaluation

This stage evaluates the selected model on the validation split and generates:

- validation metrics,
- drift report,
- pipeline summary,
- and example API payloads.

8.4 DVC Pipeline Representation

The project uses DVC to define a reproducible DAG across:

- `prepare_data`
- `train_model`
- `evaluate_model`

This supports traceable, rerunnable pipeline execution.

9 Model Development and Selection

The training process compares candidate models and selects the best one based on validation performance. The chosen configuration is then recorded in model registry metadata and used by the serving layer.

9.1 Selection Strategy

The selected model is chosen using validation results with emphasis on forecasting quality metrics, especially RMSPE.

10 Pipeline Visualization

Airflow is used to provide a DAG-level operational view of the offline ML pipeline.

10.1 Airflow DAG

The DAG visualizes the following steps:

1. prepare_data
2. train_model
3. evaluate_model

11 Experiment Tracking with MLflow

MLflow is used to record experimental runs, metrics, and model artifacts.

11.1 Tracked Information

The tracked information includes:

- model-related metrics,
- run identifiers,
- model artifacts,
- and selected best-model metadata.

11.2 Role in Reproducibility

MLflow complements DVC by capturing experiment-level traceability, while DVC captures pipeline-level stage reproducibility.

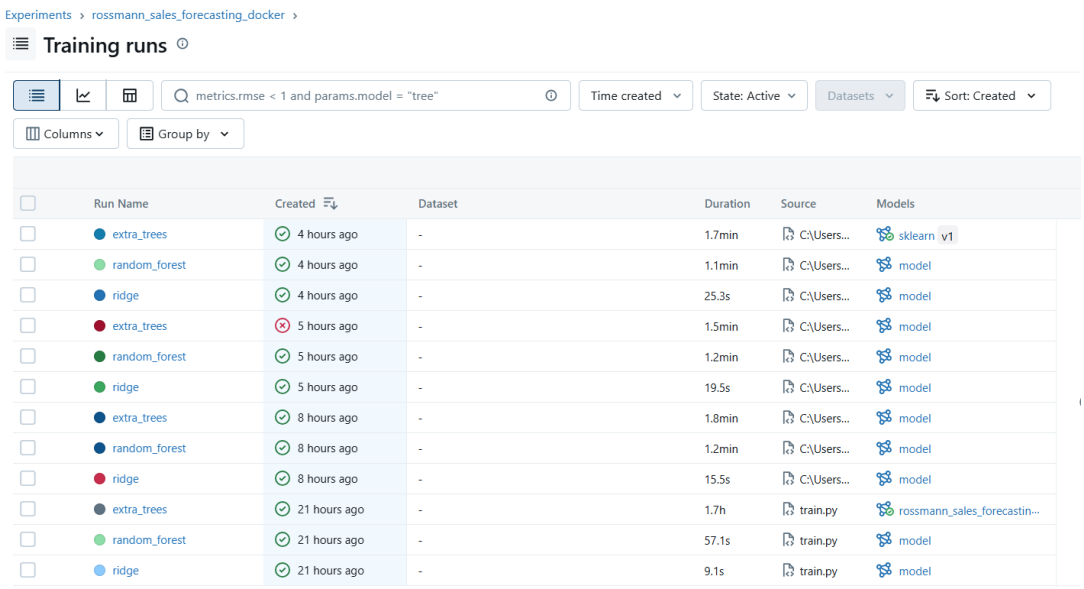

Figure 2: MLFlow Table

12 Deployment and Packaging

The application is packaged as a multi-container Docker Compose system.

12.1 Containers

The main deployment includes:

- frontend container,

- API gateway container,
- model service container,
- Prometheus container,
- Grafana container.

An additional Docker Compose file is used to start the Airflow UI stack for pipeline orchestration visibility.

12.2 Environment Parity

Containerization ensures that the serving stack is run consistently with a known dependency structure and stable port mapping.

13 API Design

The project follows a loosely coupled REST-based design.

13.1 Main API Endpoints

The gateway and serving layers expose endpoints such as:

- `GET /health`
- `GET /ready`
- `POST /forecast`
- `POST /feedback`
- `GET /pipeline`
- `GET /monitoring/summary`
- `GET /metrics`

This design ensures:

- frontend-backend independence,
- clean service boundaries,
- future replaceability of the model-serving backend,
- and easier monitoring and testing.

14 Frontend Design

The frontend is designed for non-technical users while still providing technical visibility when required.

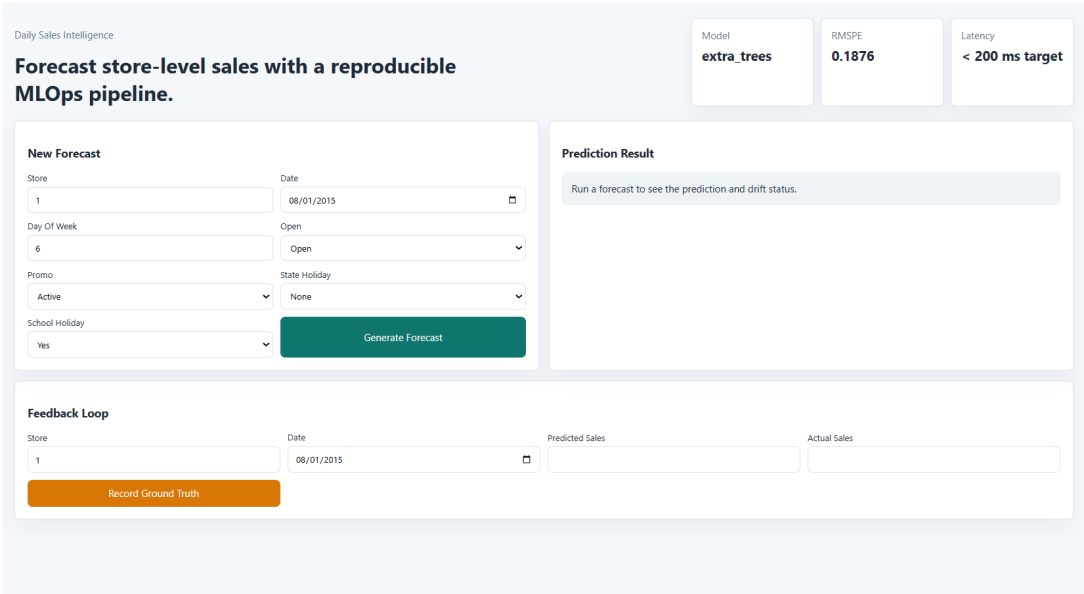

Figure 3: Frontend UI

The major screens are:

- **Forecast**: sales prediction form and result display
- **Pipeline**: pipeline summaries and selected model metadata
- **Monitoring**: operational summary for runtime status
- **User Manual**: embedded usage guidance

15 Monitoring and Alerting

Monitoring is implemented through Prometheus-based instrumentation and Grafana visualization.

Both the API gateway and model service expose metrics endpoints. Example categories of monitored information include:

- total requests,
- request latency,
- failure counts,
- feedback events,
- and drift-event counters.

15.1 Grafana Dashboard

Grafana is used as the final visualization layer in the monitoring chain. It presents runtime observability in a form suitable for system inspection and demo narration.

Figure 4: Grafana dashboard

15.2 Prometheus and Alerts

Prometheus scrapes service metrics and evaluates alert rules related to service health, failures, and drift signals.

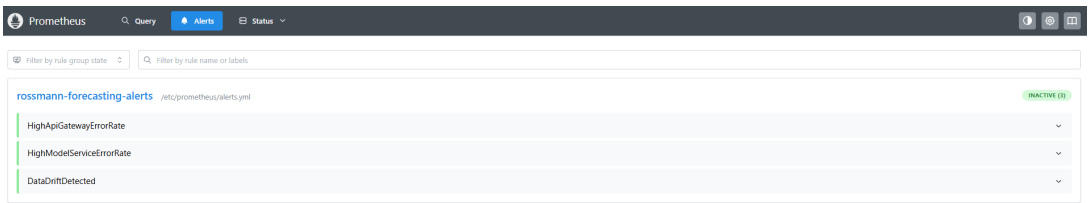

Figure 5: Prometheus alerts

16 Drift Detection and Feedback Loop

The project includes both request-level drift visibility and post-prediction feedback capture.

16.1 Drift Detection

A drift flag is generated during online inference by comparing incoming request-derived numerical feature values against previously computed baseline statistics. This provides immediate visibility when incoming patterns differ significantly from training-time behavior.

16.2 Feedback Loop

The frontend and API support logging actual sales after prediction time. This enables:

- accumulation of ground-truth data,
- live error summarization,
- and future maintenance-oriented model review.

17 Testing and Validation

The project includes test planning, implementation-level tests, and test reporting.

17.1 Testing Approach

The testing strategy covers:

- unit testing for feature logic,
- API-level validation checks,
- pipeline artifact checks,
- and service-health verification.

17.2 Acceptance-Oriented Validation

The validation flow checks that:

- the pipeline produces the required outputs,
- the services start correctly,
- requests return valid forecasts,
- and metrics are visible through the monitoring stack.

18 Conclusion

The Rossmann Sales Forecasting System with MLOps brings together the full lifecycle of a practical forecasting application. The final system includes data preparation, feature engineering, training, evaluation, experiment tracking, selected-model serving, feedback capture, monitoring, orchestration, documentation, and a non-technical user interface. The resulting solution is suitable not only for predictive demonstration but also for showcasing reproducibility, observability, and structured MLOps practice.

A Appendix A: Important URLs

- Frontend UI: http://localhost:8088
- API Gateway Docs: http://localhost:8103/docs
- Model Service Docs: http://localhost:8101/docs
- Prometheus: http://localhost:9091
- Grafana: http://localhost:3001
- Airflow: http://127.0.0.1:8081

B Appendix B: AI Usage Disclosure

AI-based tools were used as coding and writing assistants during this project. Specifically:

- Initial scaffolding and debugging support for parts of the FastAPI services and API routing.
- Assistance in structuring the DVC pipeline stages and resolving configuration issues.
- Minor help in writing Dockerfiles and Docker Compose configurations.
- Suggestions for Prometheus metric instrumentation and Grafana dashboard setup.
- Drafting and refinement of documentation and report text.

All system design decisions, pipeline structure, feature engineering logic, model training workflow, integration of components, and final validation were implemented, verified, and adapted by the author.